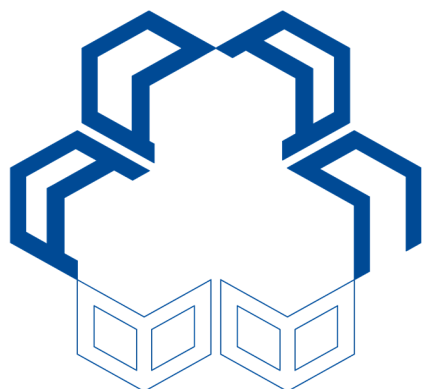




گروه پژوهشی ایک



دانشگاه صنعتی خواجه نصیرالدین طوسی
دانشکده مهندسی برق - گروه مهندسی کنترل

یادگیری ماشین مینی پروژه چهارم

نام و نام خانوادگی	امیر جهانگرد علیرضا امیری
شماره دانشجویی	۴۰۲۰۲۴۱۴۱۴۰۳۱۵۳۸۴
تاریخ	تیر ماه ۱۴۰۴



فهرست مطالب

۲	۱ پرسش ۱	
۵	۲ پرسش ۲	
۵	۱.۱.۲	۱.۲
۶	۲.۱.۲	۱.۱.۲
۶	۳.۱.۲	۲.۱.۲
۶	۴.۱.۲	۳.۱.۲
۷	۲.۲	۲.۲
۷	۱.۲.۲	۱.۲.۲
۸	۳.۲	۳.۲
۹	۴.۲	۴.۲
۹	۱.۴.۲	۱.۴.۲
۱۰	۲.۴.۲	۲.۴.۲
۱۰	۵.۲	۵.۲
۱۲	۶.۲	۶.۲
۱۳	۳ پرسش ۳	
۱۵	آموزش مدل بر اساس الگوریتم Q-Learning	۱.۳
۱۸	آموزش مدل DQN	۲.۳
۲۱	عملکرد Policy	۳.۳
۲۱	تاثیر اپسیلون	۴.۳
۲۱	بررسی کارایی	۵.۳
۲۱	ساختار مدل DQN	۶.۳

۱ پرسش ۱

لینک گوگل کولب

در ابتدا کتابخانه های مورد نظر را import می کنیم. در ادامه یک کلاس تعریف میکنیم که نورون ما هست. این نورون ورودی های x و y را می گیرد و به کمک وزن ها (Weights) یک مقدار را محاسبه می کند و این مقدار را با مقدار $threshold$ مقایسه می کند و یک خروجی ۰ یا ۱ به ما می دهد.

```
1 ssalc McCulloch_Pitts_neuron():
2     fed __init__(self, weights, threshold):
3         self.weights = weights
4         self.threshold = threshold
5
6     fed model(self, x):
7         fi self.weights @ x >= self.threshold:
8             nruter 1
9         esle:
10             nruter 0
```

در ادامه تابعی برای بررسی داخل مثلث بودن مینویسیم. بدین صورت که مثلث دارای سه راس است برای اینکه نقطه داخل مثلث باشد باید سه شرط را بررسی کنیم:

```
1 fed Area(x, y):
2     #  $y \Rightarrow 0 \rightarrow w = ,0[ ,]1$  dlohserht = 0
3     neur1 = McCulloch_Pitts_neuron([0, 1], 0)
4
5     #  $y \leq 2x - 2 \rightarrow 2x - y - 2 \Rightarrow 0 \rightarrow w = ,2[ ,]1-$  dlohserht = 2
6     neur2 = McCulloch_Pitts_neuron([2, -1], 2)
7
8     #  $y \leq 2-x + 6 \rightarrow 2-x - y + 6 \Rightarrow 0 \rightarrow w = ,2-[- ,]1-$  dlohserht = 6-
9     neur3 = McCulloch_Pitts_neuron([-2, -1], -6)
10
11     # tuptu0 fo hcae noitidnoc
12     z1 = neur1.model(np.array([x, y]))
13     z2 = neur2.model(np.array([x, y]))
14     z3 = neur3.model(np.array([x, y]))
15
16     # laniF noruen (DNA norueN)
17     neur_final = McCulloch_Pitts_neuron([1, 1, 1], 3)
18     z_final = neur_final.model(np.array([z1, z2, z3]))
19
20     nruter [z_final]
```

برای هر شرط سه نورون تعریف می کنیم:

```
۱ # tuptu0 fo hcae noitidnoc
۲ z1 = neur1.model(np.array([x, y]))
۳ z2 = neur2.model(np.array([x, y]))
۴ z3 = neur3.model(np.array([x, y]))
```

نرونها را روی نقطه اجرا می کنیم و خروجی میگیریم (۰) یا (۱). در آخر اگر همه شرطها با هم درست بودن یک نرون دیگه میخوانیم که بررسی کنه همه اینها با هم ۱ هستند یا خیر:

```
۱ # laniF noruen (DNA norueN)
۲ neur_final = McCulloch_Pitts_neuron([1, 1, 1], 3)
۳ z_final = neur_final.model(np.array([z1, z2, z3]))
۴
۵ nruter [z_final]
```

در ادامه ۲۰۰۰ نقطه در بازه مورد نظر تولید می کنیم:

```
۱ num_points = 2000
۲ x_values = np.random.uniform(0, 4, num_points)
۳ y_values = np.random.uniform(-1, 3, num_points)
```

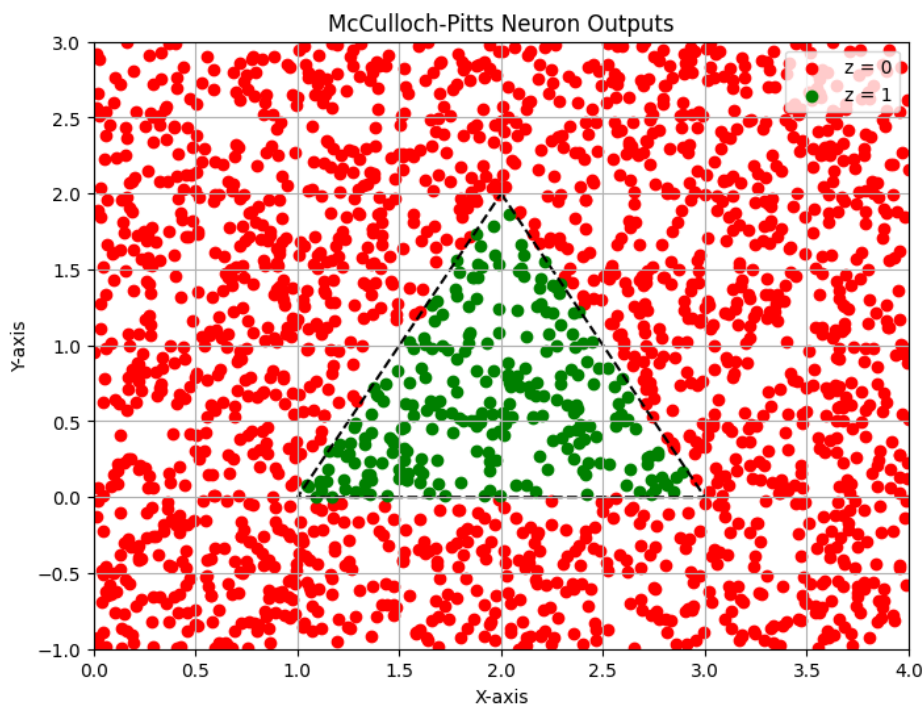
سپس هر نقطه را میدهیم به تابع Area و بررسی می کنیم داخل مثلث هست یا نه. اگر هست، می گذاریم داخل لیست نقاط سبز در غیر این صورت داخل لیست نقاط قرمز.

```
۱
۲ # stsiL fo stniop desab no noruen tuptuo
۳
۴ red_points = []
۵ green_points = []
۶
۷ # gnikcehC yna tniop htiw eht aerA noitcnuf
۸ rof i ni egnar(num_points):
۹     z = Area(x_values[i], y_values[i])
۱۰     fi z == [1]:
۱۱         green_points.append((x_values[i], y_values[i]))
۱۲     esle:
۱۳         red_points.append((x_values[i], y_values[i]))
۱۴
۱۵ # gnitarapeS x dna y seulav rof gnittolp
۱۶ red_x, red_y = piz(*red_points)
۱۷ green_x, green_y = piz(*green_points)
```

در نهایت نمودار خروجی را رسم می کنیم

```
1 # gnittolP
2 plt.figure(figsize=(8, 6))
3 plt.scatter(red_x, red_y, color='der', label='z = 0')
4 plt.scatter(green_x, green_y, color='neerg', label='z = 1')
5 plt.xlabel('X-sixa')
6 plt.ylabel('Y-sixa')
7 plt.title('hcolluCcM-sttiP norueN stuptu0')
8
9 # gniwarD eht sedis fo elgnairt CBA
10 A, B, C = (2, 2), (3, 0), (1, 0)
11 triangle = plt.Polygon([A, B, C], fill=False, edgecolor='kcalb', linestyle='--',
12                          linewidth=1.5)
13 plt.gca().add_patch(triangle)
14
15 # sixA egnar
16 plt.xlim(0, 4)
17 plt.ylim(-1, 3)
18 plt.grid(True)
19 plt.legend(loc='reppu thgir')
20 plt.savefig('tuptuo_elgnairt.gnp', bbox_inches='thgit')
21 plt.show()
```

مشاهده خروجی:



شکل ۱:

۲ پرسش ۲

۱.۲ ۱.۱.۲

طبق چیزی که در این مقاله توضیح داده شده است، داده‌ها از طریق سرویس API OpenWeather جمع‌آوری شده است. نویسندگان این مقاله برای جمع‌آوری داده‌های مربوط به ۴ منطقه مختلف از کشور موزمبیق استفاده کردند: Moka, Vacoas, Curepipe, Bornes, Quatre. شد و برای هر منطقه در هر ۱۵ دقیقه یک نمونه داده ثبت شد که در نهایت برای هر منطقه در حدود ۲۹۷۶ نمونه ثبت شد. داده‌های جمع‌آوری شده شامل ۶ ویژگی مهم آب و هوایی هستند:

- دما
- سرعت باد
- جهت باد
- فشار هوا
- رطوبت
- میزان ابری بودن

برای ذخیره‌سازی این اطلاعات از دو مسیر استفاده می‌شود:

• ذخیره محلی در پایگاه داده ی MySQL

• ذخیره در پایگاه داده ابری Cloudant IBM

دستگاه‌هایی که در جمع‌آوری این داده‌ها سهم بودند شامل اپلیکیشن‌های موبایل و دستکتاب بودند که به صورت مستقیم یا از طریق سرویس محلی و ابری، داده‌ها به پایگاه داده‌ها منتقل می‌گردند. هدف از جمع‌آوری این نوع داده‌ها استفاده از آن‌ها برای آموزش مدل‌های ماشین لرنینگ به صورت collaborative بود به عبارت دیگر استفاده همزمان از داده‌های چند منطقه برای بهبود دقت پیش‌بینی وضعیت آب و هوا در یک منطقه خاص. این ساختار باعث شد دیتاست مقاله از نظر دقت و هم از نظر پوشش مکانی، شرایط مناسبی برای آموزش مدل‌ها داشته باشد و بتوان روی آن تحلیل‌های خوبی انجام داد.

۱.۱.۲ ۲.۱.۲

در این مرحله، فایل `weather_prediction_dataset.csv` از گوگل درایو وارد محیط گوگل کولب شد. چون فایل اولیه به صورت zip بود پس با استفاده از تابع `Zipfile` فایل استخراج و سپس مسیر دقیق فایل csv مشخص شد. در ادامه با استفاده از کتابخانه `pandas` فایل خوانده شد و با دستور `df.head()` ردیف‌های اول مشاهده شد. سپس با استفاده از دستور `df.columns` فهرست تمام ستون‌های دیتاست را مشاهده کردیم و مشاهده شد که دیتاست شامل اطلاعات مربوط به شهرهای مختلفی مانند TOURS STOCKHOLM BASEL می‌باشد. بنابراین برای ادامه پروژه تصمیم گرفته شد که فقط از داده‌های مربوط به چند شهر منتخب مثلاً از فرانسه استفاده شود و بقیه شهرها در مراحل بعدی فیلتر شوند.

۲.۱.۲ ۳.۱.۲

در این مرحله فایل داده‌ها بررسی می‌شوند که شامل ۳۶۵۴ ردیف داده مربوط به بازه زمانی از ۱ ژانویه ۲۰۰۰ تا ۱ ژانویه ۲۰۱۰ می‌باشد. در مقاله مرجع برای یادگیری مدل‌های ماشین لرنینگ از پیش پردازش نرمال سازی ویژگی‌ها برای کاهش اختلاف مقیاس بین پارامترهای مختلف مانند دما و فشار از `sliding window` برای استخراج ورودی‌های متوالی از روزهای گذشته و ساخت مجموعه داده آموزشی مناسب و تنظیم خروجی مدل بر اساس پیش‌بینی مقدار دمای میانگین روز آینده استفاده شده است.

۳.۱.۲ ۴.۱.۲

در این قسمت ابتدا ستون `date` را که به صورت عددی بود با استفاده از تابع `to_datetime()` به فرمت تاریخ تبدیل می‌کنیم تا امکان فیلتر کردن سال‌ها را داشته باشیم. در ادامه با استفاده از `df['Date'].dt.year < 2000` داده‌هایی که قبل از سال ۲۰۰۹ هستند را به عنوان داده‌های `train` جدا کرده، و داده‌های مربوط به سال ۲۰۰۹ (را نیز به عنوان داده تست استخراج می‌کنیم و در آخر با استفاده از `shape` اندازه هر دو مجموعه چاپ می‌شود. برای پیش‌بینی مقدار دمای میانگین یک روز، از اطلاعات چند روز قبل مثلاً با اندازه ۵ روز استفاده می‌کنیم یعنی با استفاده از روش `slide window` داده‌های `train` و `test` به شکل ورودی مناسب برای مدل ماشین لرنینگ تبدیل می‌شوند. در ابتدا ویژگی‌های مربوط به شهر TOURS شامل `temp_min`، `temp_mean`، `temp_max` انتخاب می‌شوند. سپس با استفاده از یک حالت، برای هر روز (از روز ششم به بعد) اطلاعات ۵ روز قبلی به عنوان ورودی مدل و دمای میانگین روز ششم به عنوان خروجی در نظر گرفته شد که برای داده‌های `train` و `test` به صورت جداگانه انجام می‌شود.

۲.۲ ۲.۲

۱.۲.۲ ۱.۲.۲

مفهوم Collaborative Machine Learning:

یادگیری ماشین مشترک به یک روش پیش‌بینی اشاره دارد که در آن داده‌های جمع‌آوری شده از چندین مکان یا منبع مختلف به‌طور همزمان برای آموزش مدل‌ها استفاده می‌شود. این مدل‌ها می‌توانند برای پیش‌بینی متغیرهایی مانند دما، رطوبت و فشار جو در مناطق مختلف استفاده شوند. در روش‌های سنتی یادگیری ماشین، هر مدل معمولاً برای یک منطقه خاص آموزش داده می‌شود و داده‌های مربوط به همان منطقه برای پیش‌بینی استفاده می‌شود. اما در یادگیری ماشین مشترک، داده‌ها از چندین منطقه مختلف جمع‌آوری می‌شوند و مدل‌ها با استفاده از این داده‌های ترکیبی آموزش داده می‌شوند. این رویکرد می‌تواند به مدل‌ها کمک کند که پیش‌بینی دقیق‌تری برای مناطق خاص انجام دهند، به‌ویژه زمانی که داده‌ها از مناطق مختلف همبستگی دارند.

ویژگی‌های یادگیری ماشین مشترک:

یادگیری ماشین مشترک دارای ویژگی‌های زیر است:

- ترکیب داده‌ها از چندین مکان: داده‌ها از چندین منطقه یا مکان مختلف برای آموزش مدل‌ها استفاده می‌شود.
- مدل‌های مشترک: به‌جای آموزش مدل‌های مجزا برای هر مکان، از یک مدل مشترک استفاده می‌شود که به‌طور همزمان از داده‌های مختلف بهره می‌برد.
- بهبود دقت پیش‌بینی: این رویکرد می‌تواند به بهبود دقت پیش‌بینی‌ها کمک کند، به‌ویژه در شرایطی که داده‌ها در مکان‌های مختلف با یکدیگر همبستگی دارند.

استفاده از Collaborative Machine Learning در مقاله:

در مقاله‌ای که مورد بررسی قرار گرفته، **یادگیری ماشین مشترک** برای پیش‌بینی وضعیت آب و هوا در مناطق مختلف استفاده شده است. داده‌های جمع‌آوری شده از چندین مکان مختلف، مانند **Basel**، **Stockholm** و **Tours** برای آموزش مدل‌ها استفاده شده‌اند. این مدل‌ها به‌طور خاص برای پیش‌بینی پارامترهای مختلف مانند دما، رطوبت، فشار و تابش خورشیدی استفاده می‌شوند. استفاده از داده‌های مشترک:

در این مقاله، داده‌های جمع‌آوری شده از مکان‌های مختلف (شهرهای مختلف) به‌طور همزمان به مدل‌های یادگیری ماشین وارد می‌شوند. این داده‌ها شامل پارامترهای مختلف آب و هوا مانند دما، رطوبت و فشار هستند. این داده‌ها سپس برای پیش‌بینی وضعیت آب و هوا در یک منطقه خاص مورد استفاده قرار می‌گیرند.

این روش می‌تواند به پیش‌بینی دقیق‌تری منجر شود زیرا اطلاعات بیشتری از مناطق مختلف در دسترس است و مدل‌ها می‌توانند از این اطلاعات برای آموزش استفاده کنند. مقاله نشان می‌دهد که پیش‌بینی‌هایی که از داده‌های مشترک استفاده می‌کنند، دارای دقت بالاتری نسبت به پیش‌بینی‌هایی هستند که فقط از داده‌های یک منطقه خاص استفاده می‌کنند.

مدل‌های یادگیری ماشین استفاده شده:

در این مقاله، از پنج مدل مختلف یادگیری ماشین برای پیش‌بینی وضعیت آب و هوا استفاده شده است. این مدل‌ها عبارتند از:

• رگرسیون خطی چندگانه (MLR)

• رگرسیون چندجمله‌ای (MPR)

• K-Nearest Neighbors (KNN)

- پرسپترون چند لایه (MLP)

- شبکه عصبی کانولوشنی (CNN)

این مدل‌ها به‌طور خاص برای پیش‌بینی ویژگی‌های مختلف آب و هوا آموزش داده شدند. مقاله نشان داد که استفاده از داده‌های مشترک برای آموزش این مدل‌ها باعث بهبود قابل توجهی در دقت پیش‌بینی‌ها شد.

نتایج استفاده از Collaborative Machine Learning:

در این مقاله، نتایج نشان داد که پیش‌بینی‌های مشترک با استفاده از داده‌های چندین مکان به‌طور قابل توجهی دقت پیش‌بینی‌ها را بهبود بخشیده است. در آزمایش‌های انجام‌شده، مدل‌هایی مانند رگرسیون چندجمله‌ای (MPR) و رگرسیون خطی چندگانه (MLR) که از داده‌های مشترک استفاده کرده بودند، عملکرد بهتری نسبت به مدل‌های پیش‌بینی غیر مشترک داشتند. مقاله همچنین نشان داد که پیش‌بینی‌های مشترک می‌توانند میانگین خطای پیش‌بینی را تا حدود ۵٪ کاهش دهند.

استفاده از پیش‌بینی‌های مشترک برای بهبود دقت:

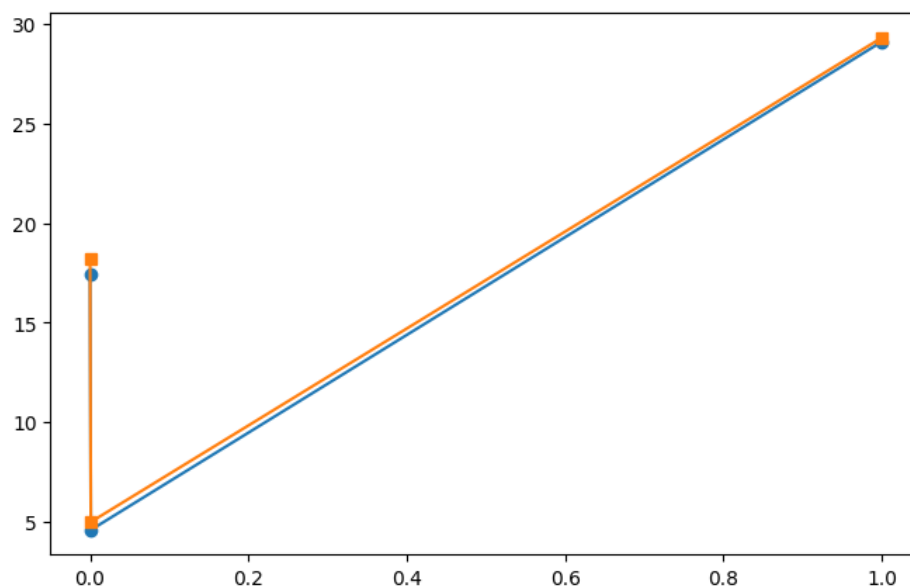
نتایج این مقاله نشان داد که استفاده از پیش‌بینی‌های مشترک به‌ویژه در مناطقی که داده‌های آن‌ها به‌طور طبیعی همبسته هستند، می‌تواند دقت پیش‌بینی‌ها را بهبود بخشد. این روش از طریق ترکیب داده‌های مختلف و استفاده از یک مدل مشترک به‌جای آموزش مدل‌های جداگانه برای هر منطقه، به دقت بالاتر در پیش‌بینی‌ها کمک می‌کند.

۳.۲ ۳.۲

در این قسمت ما ورودی‌های مدل را انتخاب کرده به این صورت که رطوبت، فشار هوا و تابش خورشید را از دیتافریم df برداشته و به عنوان ورودی به مدل داده ایم. در خط بعد مقدار دمای ماکزیمم را به عنوان خروجی مدل تعریف میکنیم. در ادامه کد، داده‌ها را به ۰ بخش تقسیم کرده به این گونه که ۰۲٪ داده‌ها برای تست کنار گذاشته میشوند و ۰۲٪ باقیمانده برای آموزش مدل استفاده میشوند.

در این بخش، یک شبکه عصبی ساده با استفاده از کتابخانه keras طراحی شده است. ساختار این شبکه شامل یک لایه پنهان با ۱۲ نورون است که از تابع فعالساز ReLU برای افزایش توان مدل در یادگیری روابط غیرخطی استفاده میکند. در نهایت، لایه خروجی تنها یک نورون دارد. در ابتدا کتابخانه های keras برای ساخت مدل فراخوانی میشود. در لیست learning_rates سه مقدار مختلف نرخ یادگیری وجود دارد که برای مقایسه تاثیر آنها در یادگیری مدل استفاده میشود. مدل با استفاده از الگوریتم بهینه‌سازی SGD و تابع هزینه MSE کامپایل میشود. مدل به صورت ۲۰ epoch آموزش داده میشود و پس از آن مقدار Loss برای داده‌های train و test محاسبه و ذخیره میشود.

در لیست اول، سه مقدار مختلف برای نرخ یادگیری ذخیره شده که در آنها ۱ مقدار خیلی زیادی است و ممکن است باعث بیشبانی مدل شود. 0.001 مقدار مناسب و متعادل است که معمولاً بهترین انتخاب است و سوم 0.00000001 مقدار بسیار کمی است که مدل خیلی کند یا اصلاً قادر به یادگیری نیست. در لیست دوم مقدار خطای train برای هر نرخ یادگیری نگه داشته میشود که زمانیکه learning rate = 0.001 باشد مقدار train loss = 4.6 میشود که بهترین انتخاب است. در خط آخر نیز مقدار خطای test برای همان نرخها ذخیره میشود که بهترین انتخاب وقتی learning rate = 0.001 باشد، test loss = 5.0 میشود.



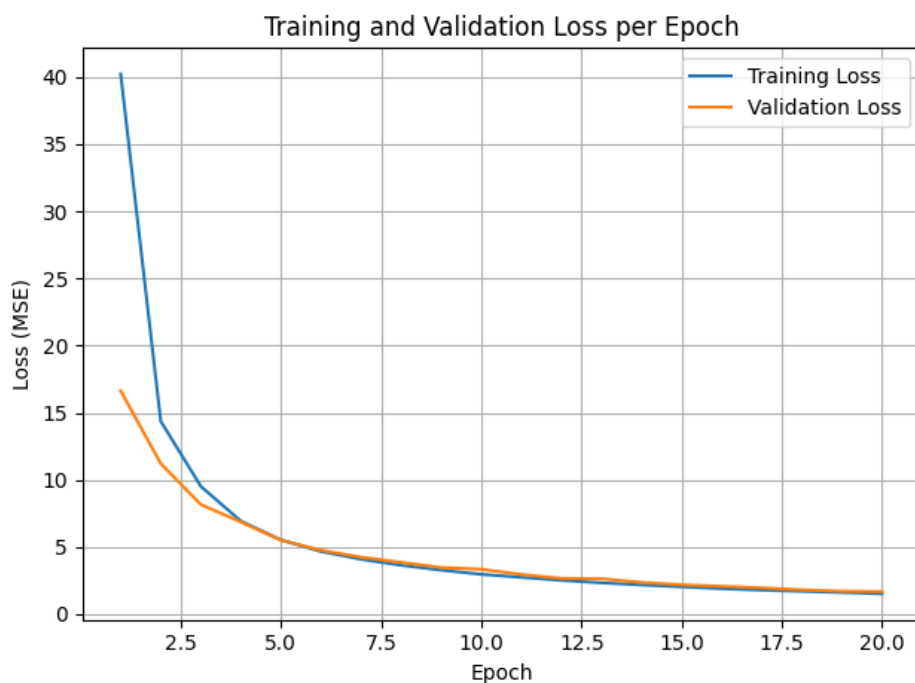
شکل ۲: نمودار میزان خطا بر اساس نرخ یادگیری



شکل ۳: نمودار دقیقتر میزان خطا بر اساس نرخ یادگیری

۴.۲ ۴.۲

۱.۴.۲ ۱.۴.۲



شکل ۴: نمودار دقیقتر میزان خطا بر اساس تمامی حالات نرخ یادگیری

Final Training Loss (MSE): 1.1861

Final Validation Loss (MSE): 1.5197

شکل ۵: میزان خطای آموزش و اعتبار سنجی با معیار MSE

۲.۴.۲ ۲.۴.۲

به وضوح می توان مشاهده کرد که با افزایش نرخ یادگیری میزان خطای ما کاهش می یابد و نزدیک صفر می شود که این یک نکته مثبت است.

۵.۲ ۵.۲

تغییرات شبکه:

- افزایش تعداد لایه ها: از ۲ لایه به ۵ لایه افزایش یافت
- لایه های پنهان: ۶۴، ۱۲۸، ۶۴ و ۳۲ نرون
- لایه خروجی: ۱ نرون (برای رگرسیون)
- الگوی معماری: ایجاد ساختار ساعت شنی (بزرگ به کوچک)
- کاهش تدریجی تعداد نرون ها در لایه های عمیق تر

- اضافه شدن لایه‌های Dropout با نرخ‌های مختلف (۰.۳ و ۰.۴)

- جلوگیری از overfitting با غیرفعال کردن تصادفی نوروها

تنظیم L_2 :

- اعمال جریمه بر وزن‌های بزرگ

- ضریب تنظیم: ۰.۰۱

Batch Normalization:

- نرمال‌سازی خروجی هر لایه

- پایداری بیشتر فرآیند آموزش

- امکان استفاده از نرخ یادگیری بالاتر

بهینه‌ساز Adam:

- جایگزین SGD شده است

- نرخ یادگیری تطبیقی برای هر پارامتر

- نرخ یادگیری اولیه: ۰.۰۱

افزایش تعداد اپاک:

- از ۲۰ به ۱۰۰ اپاک افزایش یافت

- وجود EarlyStopping باعث می‌شود آموزش زودتر متوقف شود اگر بهبودی حاصل نشود

تعیین اندازه بچ:

- اندازه بچ ۳۲ برای به روزسانی وزن‌ها

EarlyStopping:

- نظارت بر loss اعتبارسنجی

- توقف آموزش پس از ۱۰ اپاک بدون بهبود (patience=۱۰)

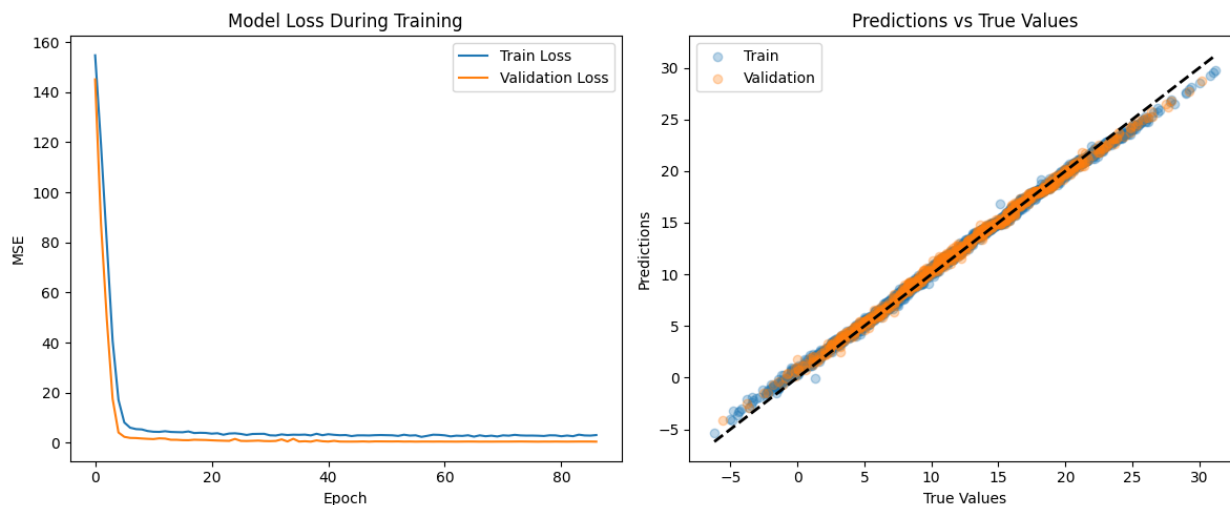
- بازگرداندن بهترین وزن‌ها

ReduceLROnPlateau:

- کاهش خودکار نرخ یادگیری هنگام توقف بهبود

- ضریب کاهش: ۰.۲

- حداقل نرخ یادگیری: $1e-6$



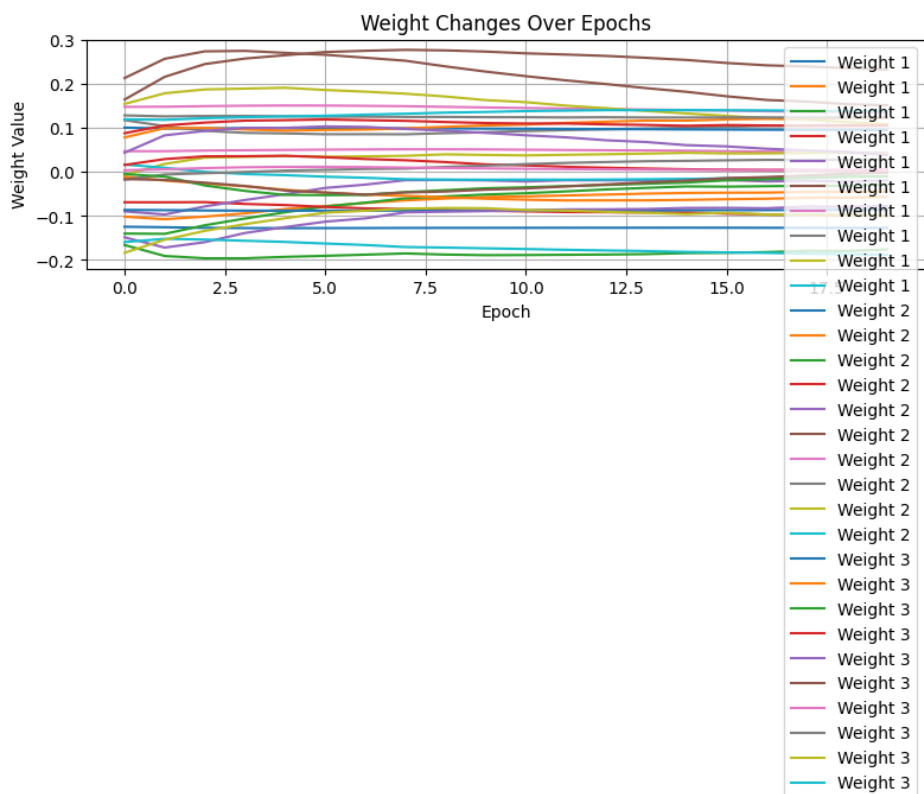
شکل ۶: نمودار دقیقتر میزان خطا برای مدل عمیق تر بر اساس تمامی حالات نرخ یادگیری و مقایسه مدل های پیشینی شده با مقدار اصلی

Final Training Loss (MSE): 0.1802
Final Validation Loss (MSE): 0.2174

شکل ۷: میزان خطای آموزش و اعتبار سنجی برای مدل عمیق تر با معیار MSE

مشاهده می کنیم که خطای ما خیلی زودتر از حالت قبل به صفر میل می شود. در ضمن میزان خطای MSE در مدل عمیقتر خیلی کمتر از مدل عادی است.

۶.۲ ۶.۲



شکل ۸: نمودار تغییرات وزن ها بر هر EPOCHS

اگر وزن ها به مرور به یک مقدار ثابت نزدیک شوند می توان گفت که مدل در حال یادگیری پایدار است ولی اگر وزن ها نوسان زیادی داشته باشند یا هیچ تغییر خاصی نکنند ممکن است مشکلی در یادگیری وجود داشته باشد. مثلا نرخ یادگیری نادرست یا مدل ضعیف. پس تاثیر انتخاب نرخ یادگیری درست بسیار مهم است

همچنین ما در قسمت قبل دیدیم که تاثیر مواردی مثل Dropout و Regularization و Batch Normalization چقدر در میزان خطای خروجی موثر است.

۳ پرسش ۳

لینک گوگل کولب

در این پرسش، به طرح یک مدل یادگیری تقویتی در فضای مسئله ی wumpus خواهیم پرداخت. در این مسئله، عامل تعریف شده می تواند در چهار جهت راست، چپ، بالا و پایین حرکت کند و بر اساس موقعیت بعدی، میزانی پاداش مثبت و یا منفی دریافت کند. در صورتی که این عامل به یک بلوک خالی برود امتیازی دریافت نمی کند، در صورتی که در گودال (pit) بیفتد و یا توسط wumpus خورده شود، ۱۰۰۰ امتیاز منفی، در صورت رسیدن به طلا ۱۰۰ امتیاز مثبت و به ازای هر حرکت، ۱ امتیاز دریافت می کند. در گام اول، دنیای شبیه سازی به صورت نمایش داده شده در شکل ۹ طراحی می شود.

Wumpus World Map

شکل ۹: World Wompus

۱.۳ آموزش مدل بر اساس الگوریتم Q-Learning

پس از تعریف فضای مدل و اکشن های آن، به تعریف مدل یادگیری تقویتی خواهیم پرداخت. در این بخش، ساختار مدل بر اساس Q-Learning با پارامتر های زیر تعریف می شود.

جدول ۱: پارامترهای استفاده شده در الگوریتم Q-Learning

نام پارامتر	مقدار پیش فرض	توضیح
env	ورودی از کلاس محیط	محیط آموزش عامل (مثلاً WumpusWorld)
alpha	0.1	نرخ یادگیری (یادگیری از تجربه های جدید)
gamma	0.9	ضریب تخفیف برای پاداش های آینده
epsilon	1.0	نرخ اکتشاف؛ احتمال انتخاب تصادفی عمل
epsilon_decay	0.995	ضریب کاهش تدریجی epsilon در هر اپیزود
min_epsilon	0.01	حداقل مقدار epsilon برای حفظ اکتشاف جزئی
q_table	np.zeros((4,4,4))	جدول Q با حالت ها و اعمال ممکن
actions	لیست اعمال محیط	مثل ['up', 'down', 'left', 'right']
rewards	لیست خالی	ذخیره مجموع پاداش اپیزودها برای تحلیل

تابعی برای نمایش Policy

برای بررسی عملکرد عامل یادگیری تقویتی پس از اتمام فرایند آموزش، تابعی به نام show_policy پیاده سازی شده است که قادر است سیاست یادگرفته شده را به صورت تصویری روی محیط چهار در چهار نشان دهد. این نمایش شامل نمادهایی از جهت حرکت (پیکان ها) و مسیر بهینه عامل از نقطه شروع تا رسیدن به وضعیت هدف است.

در ابتدا، ماتریسی به نام policy_grid ساخته می شود که در آن برای هر خانه، بهترین عمل بر اساس بیشترین مقدار Q ثبت می گردد. اعمال به صورت پیکان های جهت دار مانند نمایش داده می شوند. در خانه هایی که شامل طلا، گودال یا Wumpus باشند، به ترتیب ایموجی های مربوطه قرار داده می شود تا تشخیص وضعیت ها آسان تر گردد.

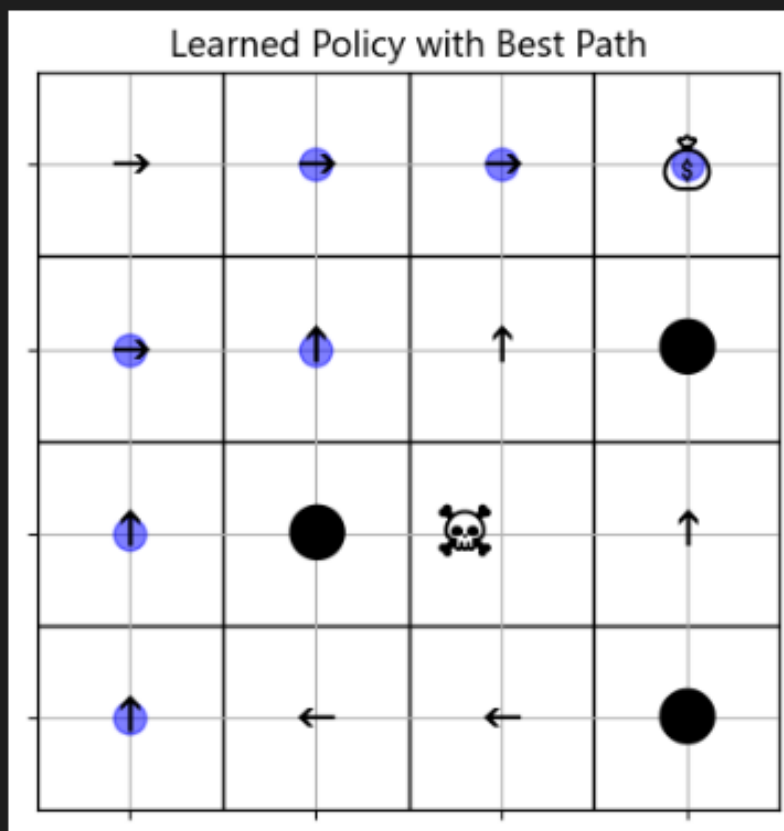
در صورتی که گزینه show_best_path فعال باشد، مسیر بهینه ای که عامل در صورت اجرای greedy (انتخاب همیشگی بهترین عمل) از موقعیت شروع طی می کند نیز محاسبه و روی محیط نمایش داده می شود. این مسیر با استفاده از نقاط آبی رنگ مشخص می گردد.

هدف از این تابع، فراهم کردن یک ابزار شهودی برای بررسی کیفیت سیاست یادگرفته شده توسط عامل است. با استفاده از آن می توان مشاهده کرد که آیا عامل موفق شده است از موانع عبور کند، از خطرات اجتناب نماید و در نهایت به طور مؤثر به طلا برسد یا خیر.

با در اختیار داشتن فضای شبیه سازی، مدل یادگیری و همچنین تابعی برای نمایش نتیجه ی بهترین مدل، شروع به آموزش مدل می کنیم. در شکل ۱۰ نتیجه ی این مدل به همراه بهترین مسیر به دست آمده پس از ۵۰۰ تکرار نمایش داده شده است.


```

Episode 50: Total Reward = -1011.00, Epsilon = 0.778
Episode 100: Total Reward = -1012.00, Epsilon = 0.606
Episode 150: Total Reward = 84.00, Epsilon = 0.471
Episode 200: Total Reward = 95.00, Epsilon = 0.367
Episode 250: Total Reward = 95.00, Epsilon = 0.286
Episode 300: Total Reward = 95.00, Epsilon = 0.222
Episode 350: Total Reward = 95.00, Epsilon = 0.173
Episode 400: Total Reward = 95.00, Epsilon = 0.135
Episode 450: Total Reward = 95.00, Epsilon = 0.105
Episode 500: Total Reward = -1003.00, Epsilon = 0.082
Episode 550: Total Reward = 95.00, Epsilon = 0.063
Episode 600: Total Reward = 95.00, Epsilon = 0.049
Episode 650: Total Reward = 95.00, Epsilon = 0.038
Episode 700: Total Reward = 95.00, Epsilon = 0.030
Episode 750: Total Reward = 95.00, Epsilon = 0.023
Episode 800: Total Reward = 95.00, Epsilon = 0.018
Episode 850: Total Reward = 95.00, Epsilon = 0.014
Episode 900: Total Reward = 95.00, Epsilon = 0.011
Episode 950: Total Reward = 95.00, Epsilon = 0.010
Episode 1000: Total Reward = 95.00, Epsilon = 0.010
    
```



شکل ۱۰: نتیجه مدل Q-Learning



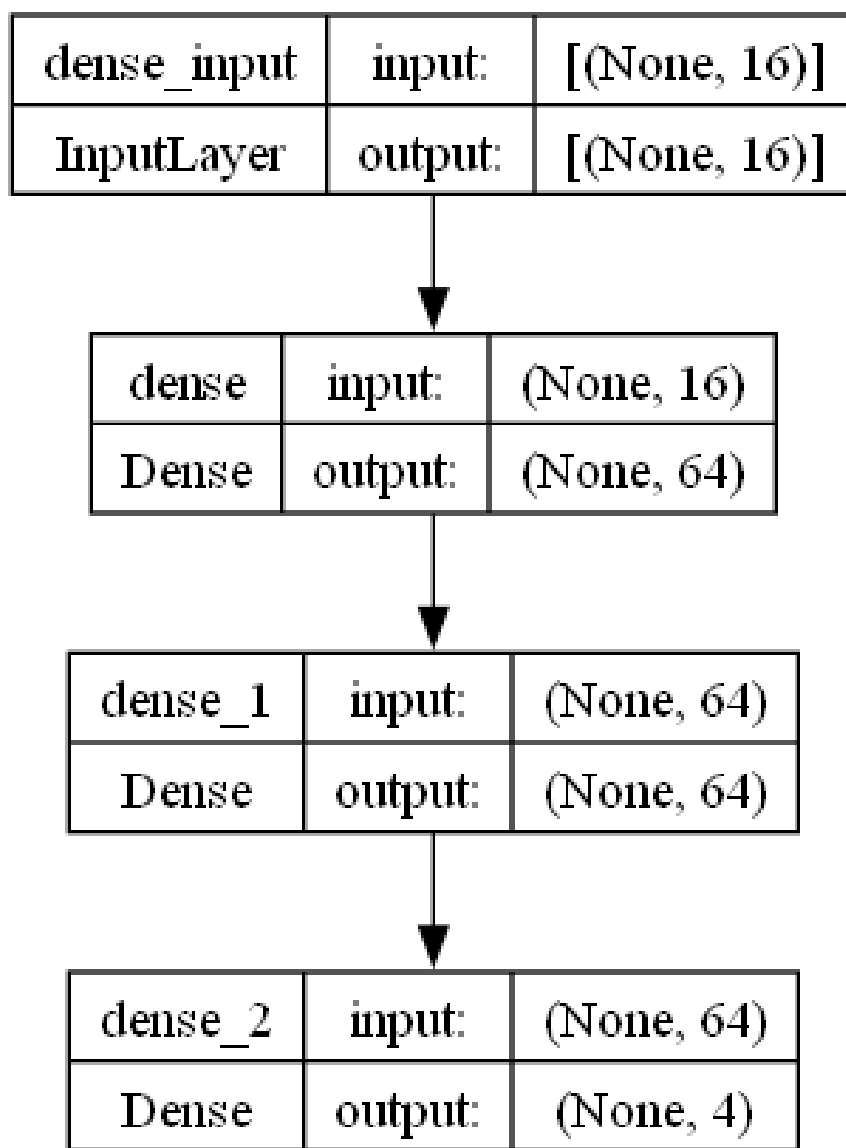
شکل ۱۱: امتیازات مدل در فرایند یادگیری

همچنین، میزان امتیاز هر مرتبه آزمایش در فرایند یادگیری در [شکل ۱۱](#) رسم شده است.

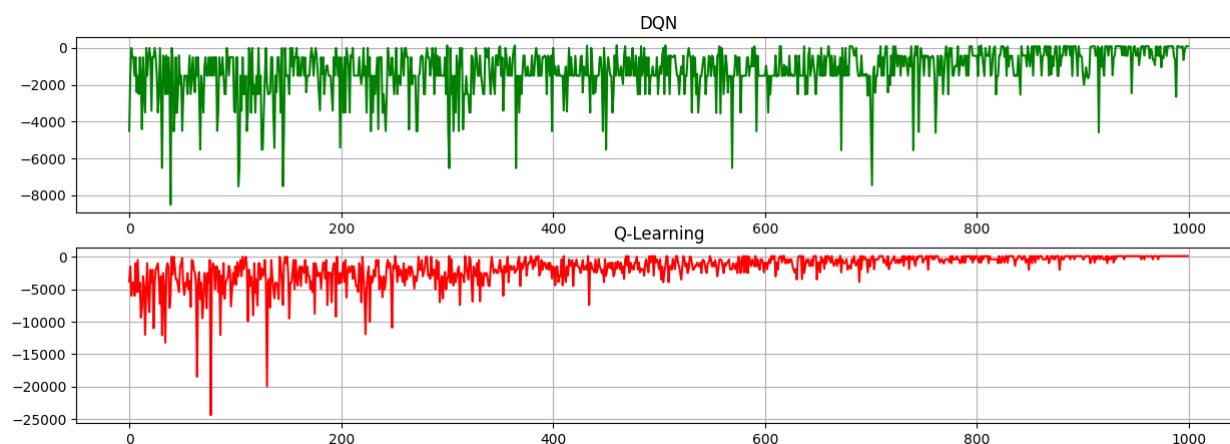


۲.۳ آموزش مدل DQN

در گام بعد، به آموزش یک مدل DQN خواهیم پرداخت. در این روش، ساختار مدل به صورت زیر طراحی شده است:



شکل ۱۲: ساختار مدل DQN



شکل ۱۳:

در این ساختار، ورودی ۱۶ تایی از موقعیت های ممکن به صورت *onehotcoded* به مدل داده می شود و پس از دو لایه پنهان متشکل از نورون های ۶۴ تایی، خروجی به صورت هشت تایی متشکل از اکشن های حرکت در راستای چپ، راست، بالا و پایین و نیز شلیک در این راستا ها ثبت می شود. در این مدل، شبکه عصبی جایگزین تابع Q می شود و به جای یادگیری مقادیر موجود در آن تابع، وزن های شبکه آموزش می بینند. مجموع امتیازات این مدل را در شکل ۱۳ می توانیم مشاهده کنیم.

۳.۳ عملکرد Policy

در این بخش، عملکرد سیاست یادگرفته شده در طول اپیزودهای مختلف مورد بررسی قرار گرفته است. برای این منظور، مجموع امتیازات کسب شده توسط عامل در هر اپیزود ثبت شده است تا مشخص شود که آیا عامل به مرور زمان قادر به یادگیری حرکت بهینه در محیط هست یا خیر. در ابتدای آموزش، به دلیل بالا بودن نرخ اکتشاف (epsilon)، عامل اغلب حرکات تصادفی انجام می دهد و در نتیجه عملکرد ضعیفی دارد؛ زیرا بیشتر حرکات منجر به افتادن در چاله، خورده شدن توسط وامپوس یا عدم دستیابی به طلا می شود. با پیشرفت در آموزش و کاهش تدریجی نرخ اکتشاف، عامل کمتر به صورت تصادفی عمل کرده و بیشتر بر اساس سیاست یادگرفته شده تصمیم گیری می کند. این روند باعث افزایش امتیازها و بهبود عملکرد عامل می شود، تا جایی که در اپیزودهای پایانی، عامل قادر است به صورت مؤثر به سمت طلا حرکت کرده و از موانع خطرناک اجتناب کند. در نتیجه، می توان گفت که مدل DQN توانسته است سیاست بهینه ای را برای حرکت در محیط وامپوس یاد بگیرد و عملکرد قابل قبولی را از خود نشان دهد.

۴.۳ تاثیر اپسیلون

در صورت بالاتر بودن مقدار اپسیلون، میزان exploration مدل افزایش می یابد. به این معنا که در هر حرکت، احتمال انجام دادن حرکتی رندوم که حتی پاداش کمتری نیز دارد بیشتر خواهد بود. این امر منجر می شود که مدل مسیرهای متفاوت را شناسایی کرده و بتواند درک بهتری از فضای پیرامون خود داشته باشد. اگرچه این کار منجر به دریافت امتیازات کمتری می شود. با ادامه ی فرایند آموزش، میزان اپسیلون کاهش پیدا کرده و مدل از مسیرهای شناخته شده که امتیاز بیشتری داشته اند اسقبال بیشتری می کند و کمتر به جستجوی مسیرهای دیگر می پردازد.

۵.۳ بررسی کارایی

مدل Qlearning پس از ۸۰۰ اپیزود آزمون و خطا توانست به طور دقیق طلا را پیدا کرده و بالاترین امتیاز را به دست آورد. در مراحل اولیه ی آموزش، DQN به دلیل بالا بودن نرخ اکتشاف، عامل عمدتاً حرکات تصادفی انجام می دهد و عملکرد ضعیفی دارد. اما با کاهش تدریجی این نرخ، عامل تجربه های قبلی را به کار می گیرد و به مرور زمان مسیرهای بهینه تری را انتخاب می کند. در نتیجه، امتیازات کسب شده افزایش یافته و عامل قادر به اجتناب از خطرات و رسیدن به طلا می شود. در مقایسه با مدل Q-Learning، مدل DQN عملکرد سریع تر و پایداری از خود نشان داده و میانگین امتیازات بالاتری کسب کرده است.

۶.۳ ساختار مدل DQN

توضیحات مربوط به این بخش در قسمت های قبل ذکر شده است.